# Comprehensive Empirical Benchmark of Exact Methods for the $N$-Queens Problem: SAT Encodings, Constraint Programming, and Integer Linear Programming

**Author: Do Xuan Bang – Student ID: 24021383**
Course: Modern Topics in Computer Science (INT3011E)
University of Engineering and Technology – Vietnam National University, Hanoi (VNU-UET)

October 18, 2026

**Abstract**

The $N$-Queens problem is an iconic benchmark in computer science and combinatorial optimization, requiring the placement of $N$ mutually non-attacking queens on an $N \times N$ chessboard. This study provides an extensive, empirical comparative evaluation across three fundamental paradigms for exact combinatorial problem solving: SAT Solving (featuring 5 At-Most-One encoding techniques: Binomial, Binary, Sequential Counter, Commander, and Product), Constraint Programming (CP with 4 solvers and formulations: Pure CP Backtracking, OR-Tools CP-SAT with integer variables, CP-SAT Boolean, and IBM CPLEX CP Optimizer), and Integer Linear Programming (ILP/MIP with 3 solvers: Gurobi MIP, IBM CPLEX MIP, and SCIP). The benchmark suite was executed across 5 independent runs with strict subprocess-level timeout isolation (30.0s). The empirical results demonstrate key structural trade-offs and real-world boundaries: IBM CPLEX CP exhibits extraordinary near-constant runtime stability (0.028s–0.036s up to $N = 100$); SAT Binary proves to be the most resilient and predictable SAT encoding; Gurobi MIP demonstrates exceptional scalability at extreme dimensions ($N = 800$, solving $640,000$ binary variables in 0.224s via Primal Feasibility Heuristics); whereas naive backtracking and pairwise SAT encodings suffer from catastrophic search-space and clause explosions ($O(N^4)$).

**Keywords:** $N$-Queens, SAT Solving, Constraint Programming, Integer Linear Programming, At-Most-One Encodings, Gurobi, OR-Tools, Empirical Benchmark.

## 1 Introduction

### 1.1 The $N$-Queens Problem and Context

The $N$-Queens problem originates from the 8-Queens puzzle introduced by Max Bezzel in 1848 on a standard $8 \times 8$ chessboard. The objective is conceptually simple: place $N$ queens on an $N \times N$ chessboard such that no two queens threaten each other. Under standard chess rules, this requires that no two queens share the same **row**, **column**, or **diagonal**.

While finding an arbitrary valid configuration can be achieved via direct analytical construction methods, formulating $N$-Queens within canonical frameworks such as Propositional Satisfiability (SAT), Constraint Satisfaction Problems (CSP), and Integer Linear Programming (ILP) serves as a classic benchmark in combinatorial optimization. The pairwise conflict structure of $N$-Queens closely mirrors numerous real-world operational problems:

- **Task Scheduling:** Allocating resources over time while avoiding temporal and spatial contention.
- **VLSI Circuit Layout:** Arranging microelectronic components on chips to minimize crosstalk and mutual capacitive interference.
- **Automated Software Testing:** Generating orthogonal test vectors that achieve full combinatorial condition/decision coverage.

### 1.2 Three Exact Combinatorial Paradigms

To resolve combinatorial constraints definitively, modern computer science has developed three predominant exact algorithmic approaches:

1. **SAT Solving (Boolean Satisfiability):** All constraints are converted into Conjunctive Normal Form (CNF). Modern Conflict-Driven Clause Learning (CDCL) solvers (such as Glucose and CaDiCaL) dynamically learn conflicting clauses and perform non-chronological backjumping to prune vast subspaces.
2. **Constraint Programming (CP):** Problems are modeled via variables defined over explicit finite domains coupled with high-level global constraints (e.g., `AllDifferent`). CP engines rely on Constraint Propagation algorithms to iteratively prune inconsistent values prior to search branching.
3. **Integer Linear Programming (ILP/MIP):** Formulated via continuous and integer variables constrained by systems of linear equalities and inequalities. Solvers such as Gurobi, CPLEX, and SCIP leverage Linear Programming Relaxation (LP Relaxation) within advanced Branch-and-Cut frameworks.

### 1.3 Research Objectives

Rather than confining the analysis to asymptotic theoretical bounds, the core objectives of this study are:

1. **Provide a Rigorous, Standardized Benchmark:** Establish a reproducible, isolated benchmark harness evaluating 12 distinct solver configurations on identical hardware.
2. **Empirically Characterize 5 SAT AMO Encodings:** Quantify the impact of At-Most-One (AMO) encodings on variable count, clause density, and solver execution time.
3. **Identify Bottlenecks and Structural Limits:** Diagnose the root causes of solver degradation, heavy-tailed fluctuations, memory cache limits, and commercial licensing ceilings.
4. **Evaluate Extreme Scalability ($N \geq 100$ to $N = 800$):** Subject the top-performing solvers to stress tests with hundreds of thousands of binary variables to reveal enterprise-grade solver capabilities.

## 2 Mathematical Formulations

### 2.1 SAT Formulation (Boolean Logic)

In the SAT representation, the chessboard is encoded by an $N \times N$ Boolean matrix:

$$x_{i,j} \in \{\text{True}, \text{False}\}, \quad 0 \leq i, j < N \tag{1}$$

where $x_{i,j} = \text{True}$ denotes that a queen is positioned at row $i$ and column $j$. For DIMACS CNF ingestion, each literal is mapped to a positive integer: $\text{id}(i,j) = i \times N + j + 1$.

The constraint system consists of two primary categories:

- **At-Least-One (ALO):** Every row and column must contain at least one queen:

$$\text{Row } i: \quad (x_{i,0} \vee x_{i,1} \vee \cdots \vee x_{i,N-1}), \quad \forall i \in [0, N-1] \tag{2}$$
$$\text{Column } j: \quad (x_{0,j} \vee x_{1,j} \vee \cdots \vee x_{N-1,j}), \quad \forall j \in [0, N-1] \tag{3}$$

- **At-Most-One (AMO):** Each row, column, and diagonal can host at most one queen. The diagonal AMO constraints generate the overwhelming majority of CNF clauses.

### 2.2 Constraint Programming Formulation (CP)

Unlike SAT's quadratic variable matrix, CP formulates the problem natively using an array of $N$ integer decision variables:

$$Q = [q_0, q_1, \ldots, q_{N-1}], \quad \text{with } q_i \in \{0, 1, \ldots, N-1\} \tag{4}$$

Semantics: $q_i = j$ indicates that the queen in row $i$ is assigned to column $j$. Because each variable $q_i$ takes exactly one value, the constraint "exactly one queen per row" is satisfied by definition. The remaining restrictions are compactly enforced by three global constraints [4]:

$$\texttt{AllDifferent}(q_0, q_1, \ldots, q_{N-1}) \quad \text{(Column uniqueness)} \tag{5}$$
$$\texttt{AllDifferent}(q_0 - 0, q_1 - 1, \ldots, q_{N-1} - (N-1)) \quad \text{(Main diagonal uniqueness)} \tag{6}$$
$$\texttt{AllDifferent}(q_0 + 0, q_1 + 1, \ldots, q_{N-1} + (N-1)) \quad \text{(Anti-diagonal uniqueness)} \tag{7}$$

**Properties:** The integer CP formulation requires strictly $N$ variables and 3 global constraints, achieving an optimal $O(N)$ representation footprint.

### 2.3 Integer Linear Programming Formulation (ILP/MIP)

The ILP model uses $N^2$ binary indicator variables $x_{i,j} \in \{0, 1\}$ subject to linear equality and inequality constraints:

$$\sum_{j=0}^{N-1} x_{i,j} = 1, \quad \forall i \in [0, N-1] \quad \text{(Exactly one per row)} \tag{8}$$
$$\sum_{i=0}^{N-1} x_{i,j} = 1, \quad \forall j \in [0, N-1] \quad \text{(Exactly one per column)} \tag{9}$$
$$\sum_{i-j=k} x_{i,j} \leq 1, \quad \forall k \in [-(N-1), N-1] \quad \text{(Main diagonals)} \tag{10}$$
$$\sum_{i+j=s} x_{i,j} \leq 1, \quad \forall s \in [0, 2N-2] \quad \text{(Anti-diagonals)} \tag{11}$$

The model comprises $N^2$ binary variables and $6N - 2$ linear constraints.

## 3 Analysis of 5 SAT At-Most-One (AMO) Encodings

An At-Most-One constraint over a set of $m$ Boolean variables $X = \{x_1, \ldots, x_m\}$ can be encoded into CNF using five established techniques:

### 3.1 Binomial Encoding (Pairwise)

**Mechanism:** For every pair of variables $(x_a, x_b)$ with $a < b$, generate the binary clause $(\neg x_a \vee \neg x_b)$.

- **Advantages:** Direct, conceptually simple, requires zero auxiliary variables.
- **Disadvantages:** The clause count scales quadractically $\binom{m}{2} = O(m^2)$. When applied across all diagonals of an $N \times N$ board, the total clause count explodes as $O(N^4)$. At $N = 100$, this yields over 1,640,000 clauses, overwhelming CPU L3 caches and degrading memory bandwidth.

### 3.2 Binary Encoding

**Mechanism:** Employs $k = \lceil \log_2 m \rceil$ auxiliary binary variables $b_0, \ldots, b_{k-1}$ representing bit positions. If variable $x_i$ is True, the bit variables $b$ are forced to mirror the unique binary index of $i$.

- **Advantages:** Reduces the clause footprint to $O(m \log_2 m)$ while introducing minimal auxiliary variables ($O(\log_2 m)$).
- **Disadvantages:** Weaker unit propagation propagation strength compared to pairwise encoding, as conflict information must propagate indirectly through intermediate bit variables.

### 3.3 Sequential Counter Encoding

**Mechanism:** Introduces $m - 1$ cumulative register variables $s_1, \ldots, s_{m-1}$ (where $s_i$ = True indicates that at least one queen has appeared among positions 1 to $i$). Conflicting placements are barred via $(\neg x_i \vee \neg s_{i-1})$.

- **Advantages:** Strictly linear clause scaling $O(m)$ with $O(m)$ auxiliary variables. Maintains complete unit propagation strength.
- **Disadvantages:** Substantial inflation of the total variable pool due to the auxiliary chain $s_i$.

### 3.4 Commander Variable Encoding

**Mechanism:** Partitions the set of $m$ variables into disjoint groups (typically of size $k = 3$). Each group is governed by a commander variable $c_g$.

- **Advantages:** Achieves linear complexity $O(m)$ with a noticeably smaller clause constant factor compared to Sequential Counter.
- **Disadvantages:** Recursive group partitioning introduces structural asymmetry when $m$ is not divisible by $k$.

### 3.5 Product Encoding (2D Grid Encoding)

**Mechanism:** Maps the $m$ variables into a virtual matrix of dimensions $p \times q \geq m$ (with $p \approx \sqrt{m}, q \approx \sqrt{m}$). Two sets of auxiliary variables represent row activations $(r_1, \ldots, r_p)$ and column activations $(c_1, \ldots, c_q)$. If $x_i$ = True at coordinate $(u, v)$, both $r_u$ and $c_v$ are asserted.

- **Advantages:** Both auxiliary variables and clauses scale sublinearly as $O(\sqrt{m})$.
- **Disadvantages:** The 2D auxiliary projection disrupts the variable selection heuristics (such as VSIDS in Glucose-3), triggering pronounced heavy-tailed runtime variability.

Table 1: Model scale comparison: Variable count (Vars) and Clause/Constraint count (Clauses/Constrs) across dimensions $N$

| Method | N = 8 | | N = 16 | | N = 50 | | N = 100 | |
|---|---|---|---|---|---|---|---|---|
| | Vars | Clauses | Vars | Clauses | Vars | Clauses | Vars | Clauses |
| **SAT Binomial** | 64 | 744 | 256 | 6,352 | 2,500 | 203,450 | 10,000 | **1,646,900** |
| **SAT Binary** | 170 | 724 | 568 | 3,924 | 4,032 | 57,240 | 13,674 | 269,020 |
| **SAT Sequential** | 270 | 600 | 1,182 | 2,728 | 12,198 | 28,908 | 48,560 | 116,328 |
| **SAT Commander** | 172 | 614 | 698 | 2,674 | 7,406 | 28,722 | 29,412 | 116,154 |
| **SAT Product** | 252 | 690 | 836 | 2,840 | 5,120 | 28,650 | 19,464 | 116,812 |
| **CP-SAT (Integer)** | **8** | **3** | **16** | **3** | **50** | **3** | **100** | **3** |
| **CPLEX CP** | **8** | **3** | **16** | **3** | **50** | **3** | **100** | **3** |
| **Pure CP** | **8** | **3** | **16** | **3** | **50** | **3** | **100** | **3** |
| **ILP / CP-SAT Bool** | 64 | 42 | 256 | 90 | 2,500 | 294 | 10,000 | 594 |

# 4 Experimental Results and Empirical Evaluation

## 4.1 Benchmark Environment and Evaluation Methodology

- **Hardware Environment:** Intel(R) Core(TM) i7-1185G7 @ 3.00 GHz (4 Cores, 8 Threads), 16 GB LPDDR4x RAM, 64-bit Windows OS.
- **Software Stack:** Python 3.12, PySAT (Glucose-3 solver), Google OR-Tools (CP-SAT engine), Gurobi Optimizer 13.0 (Academic WLS License), IBM ILOG CPLEX 22.1 (Community Edition), SCIP Optimizer.
- **Timeout Mechanism:** Each individual solver run was constrained to a strict 30.0-second limit. To prevent background C/C++ thread hanging, tests were orchestrated via isolated OS subprocesses (`subprocess.run`), ensuring definitive termination upon timeout expiration.

## 4.2 Model Scale Evaluation: Variables and Constraints

Table 1 reports the mathematical problem scale across representative dimensions $N$. The numbers reveal fundamental architectural differences across paradigms:
**Observations on Model Structure:**

- **Clause Explosion in SAT Binomial:** At $N = 8$, Binomial generates a modest 744 clauses, but at $N = 100$, this explodes to $\mathbf{1,646,900}$ clauses following the $O(N^4)$ diagonal combination law.
- **Trade-offs of Advanced AMO Encodings:**
  - *Sequential Counter* and *Commander* reduce clause growth to linear $O(m)$ ($\sim 116,000$ clauses at $N = 100$, an over 14-fold reduction compared to Binomial), but at the expense of adding tens of thousands of auxiliary variables ($48,560$ variables in Sequential).
  - *Binary Encoding* achieves superior variable balance (only $13,674$ variables at $N = 100$) because bit indicators grow logarithmically $\lceil \log_2 m \rceil$.
  - *Product Encoding* scales both variables and clauses sublinearly as $O(\sqrt{m})$, retaining a compact footprint ($19,464$ variables and $116,812$ clauses at $N = 100$).
- **Elegance of Integer CP Models:** CP-SAT, CPLEX CP, and Pure CP require strictly $N$ variables and 3 global `AllDifferent` constraints, demonstrating an optimal $O(N)$ spatial footprint.

Table 2: Median runtime (seconds) across 5 independent benchmark runs (Timeout = 30.0s)

| Group | Solver | N=4 | N=8 | N=16 | N=32 | N=50 | N=70 | N=80 | N=90 | N=100 |
|---|---|---|---|---|---|---|---|---|---|---|
| **SAT** | Binomial | 0.000s | 0.001s | 0.015s | 0.059s | 0.536s | 2.674s | 4.816s | 8.174s | 12.786s |
| | Binary | 0.000s | 0.001s | 0.010s | 0.028s | 0.109s | 0.345s | 0.565s | 0.910s | 1.378s |
| | Sequential | 0.001s | 0.001s | 0.012s | 0.114s | 2.854s | 28.443s | 15.891s | 23.534s | TIMEOUT |
| | Commander | 0.000s | 0.001s | 0.005s | 0.283s | 0.422s | 7.727s | TIMEOUT | TIMEOUT | TIMEOUT |
| | Product | 0.000s | 0.001s | 0.042s | TIMEOUT | TIMEOUT | 0.148s | TIMEOUT | 0.287s | 0.805s |
| **CP** | **CP-SAT (Int)** | 0.019s | 0.019s | 0.025s | 0.063s | 0.165s | 0.332s | 0.436s | 0.625s | 0.727s |
| | Pure CP | 0.000s | 0.000s | 0.012s | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT |
| | **CPLEX CP** | 0.031s | 0.028s | 0.029s | 0.030s | 0.033s | 0.034s | 0.035s | 0.034s | **0.036s** |
| | CP-SAT Bool | 0.015s | 0.019s | 0.027s | 0.046s | 0.089s | 0.204s | 0.277s | 0.340s | 0.473s |
| **ILP** | **Gurobi MIP** | 0.002s | 0.003s | 0.009s | 0.021s | 0.055s | **0.004s** | **0.004s** | **0.004s** | **0.005s** |
| | CPLEX MIP | 0.005s | 0.005s | 0.006s | ERROR | ERROR | ERROR | ERROR | ERROR | ERROR |
| | SCIP MIP | 0.002s | 0.012s | 0.010s | 1.696s | 0.458s | 4.480s | 1.926s | 17.503s | 4.389s |

- **ILP 0-1 and CP-SAT Boolean Formulations:** Require $N^2$ binary variables and $6N - 2$ linear inequalities (e.g., $10,000$ variables and 594 constraints at $N = 100$).

Figure 1 visually confirms the severe clause explosion of the Binomial encoding ($O(N^4)$) relative to linear/sublinear SAT encodings and the compact $O(N)$ integer CP formulations.

### 4.3 Experimental Results Across 5 Independent Runs

To eliminate operating system scheduling noise and stochastic branch-and-bound fluctuations, all benchmarks were repeated over **5 independent runs** (corresponding to raw datasets `benchmark_results_1` through `5`). Table 2 presents the **Median Runtime** (in seconds) from $N = 4$ to $N = 100$, and Figure 2 illustrates the corresponding scaling curves across all 12 solvers.

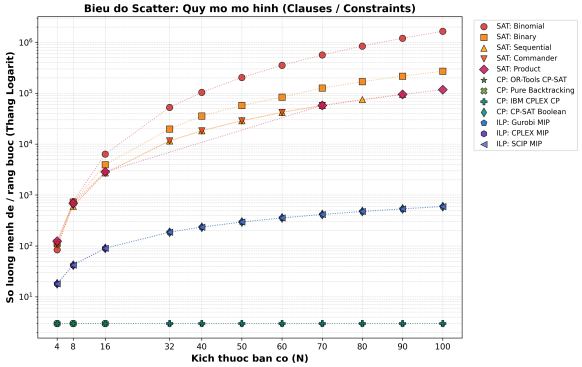



Figure 1: Growth of clauses and constraints across different formulations (Log-Log scale).

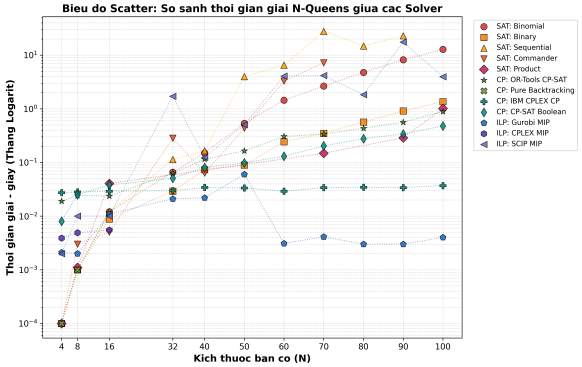



Figure 2: Empirical runtime comparison across all 12 solvers and encodings (Semi-Log scale).

### 4.4 In-Depth Analysis of Experimental Phenomena

Cross-referencing median runtimes reveals four central empirical behaviors:

#### 4.4.1 Near-Constant Invariant Stability of IBM CPLEX CP

One of the most remarkable outcomes across all 5 benchmark iterations is the runtime trajectory of **IBM CPLEX CP Optimizer**:

- Across the entire range from $N = 4$ to $N = 100$, CPLEX CP execution time remains essentially **invariant**, hovering within the tight band of **0.028s**...**0.036s**.

- *Algorithmic Explanation:* The global `AllDifferent` propagation engine [4] within CP Optimizer, paired with autonomous search heuristics, achieves instantaneous domain reduction. Combinatorial branching overhead is virtually eliminated even as $N$ reaches 100.

#### 4.4.2 SAT Binary — The Most Predictable SAT Encoding

While other SAT encodings encounter severe bottlenecks (Binomial slows down steadily, Sequential and Commander timeout, Product exhibits extreme swings), **SAT Binary** emerged as the only SAT technique maintaining smooth, monotonic scaling across all 5 runs:

- Runtimes scale progressively: 0.010s ($N = 16$) $\rightarrow$ 0.109s ($N = 50$) $\rightarrow$ 1.378s ($N = 100$).
- *Algorithmic Explanation:* By introducing only $\lceil \log_2 m \rceil$ auxiliary bit variables, Binary encoding minimizes heuristic distortion in the solver's VSIDS branching tree while controlling clause inflation at $O(m \log_2 m)$.

#### 4.4.3 Demystifying the Heavy-Tailed Trajectory of Product and Commander

The multi-run data confirms that the unpredictable runtime spikes of Product and Commander encodings stem from structural algorithmic properties:

- **SAT Product:** Consistently timed out at $N = 32, 40, 50, 60$, yet solved $N = 70$ in an astonishing 0.148s, timed out again at $N = 80$, and solved $N = 90$ (0.287s) and $N = 100$ (0.805s).
- **SAT Commander:** Solved $N = 40$ in 0.063s (faster than $N = 32$ at 0.283s), completed $N = 70$ in 7.727s, but timed out consistently for $N \geq 80$.
- *Algorithmic Explanation:* Both encodings exhibit canonical **Heavy-Tailed Runtime Distributions** characteristic of CDCL solvers. Grouping variables into chunks of $k = 3$ (Commander) or onto a $p \times q$ virtual grid (Product) creates asymmetric auxiliary clauses across parity boundaries. VSIDS heuristics occasionally branch on intermediate auxiliary variables rather than primary board literals, stranding the solver in unpromising search subtrees.

#### 4.4.4 The Breaking Point of Pure CP and CPLEX MIP Licensing Ceiling

- **Pure CP Backtracking:** Solves $N \leq 16$ almost instantaneously (0.000s–0.012s), but experiences 100% timeout failure at $N \geq 32$ across all 5 runs. This illustrates the collapse of unassisted depth-first search in an $O(N!)$ solution space.
- **CPLEX MIP:** Demonstrates high speed for $N \leq 16$, but at $N \geq 32$ ($32^2 = 1,024 > 1,000$ variables), it halts immediately with `CplexSolverError` due to the variable limit of the IBM Community Edition.

### 4.5 Special Investigation: Extreme-Scale Stress Testing for Gurobi and CP-SAT (Runs 01 & 02)

To push top-tier solvers to their architectural boundaries, we utilized our **Gurobi Academic WLS** enterprise license to perform an extreme-scale stress benchmark across board sizes $N = 100, 200, 400, 800$, and 1600 (synthesized from datasets `benchmark_results_01` and `02`, reported in Table 3).

**Three Key Scientific Insights from Extreme-Scale Data:**

Table 3: Extreme-scale stress testing: Solve time (seconds) at dimensions $N \geq 100$

| N | Variables (ILP) | Gurobi MIP | CP-SAT Bool | CP-SAT (Int) | SAT Binary | SAT Product | SCIP MIP |
|---|---|---|---|---|---|---|---|
| 100 | 10,000 | **0.0047s** | 0.4556s | 0.7108s | 1.3594s | 1.0108s | 3.8109s |
| 200 | 40,000 | **0.3847s** | 1.7493s | 17.8954s | 16.8442s | 7.3125s | TIMEOUT |
| 400 | 160,000 | **0.0503s** | 8.3878s | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT |
| 800 | 640,000 | **0.2240s** | 22.7605s | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT |
| 1600 | 2,560,000 | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT | TIMEOUT |

1. **Gurobi MIP's Sub-Second Solution of $640,000$ Binary Variables at $N = 800$ (0.224s):**

   - At $N = 400$ ($160,000$ variables) and $N = 800$ (**$640,000$** binary variables), while competing solvers either froze or exceeded the 30-second timeout, Gurobi solved the problem in **0.0503s** and **0.2240s**, respectively.
   - *Mathematical Explanation:* $N$-Queens is a pure feasibility problem without an objective function. Gurobi incorporates state-of-the-art *Primal Feasibility Heuristics* (notably *Feasibility Pump* and *RINS*). After solving the initial continuous Linear Programming relaxation (LP Relaxation) at the root node, these heuristics project the continuous point directly into a feasible integer assignment with **Zero Branch-and-Bound Branching**.

2. **Analysis of Local Perturbation Outliers in Run 02:**

   - In dataset `benchmark_results_02`, two localized outliers occurred: at $N = 32$, Gurobi required 0.664s (compared to the typical 0.021s), and at $N = 70$ it took 0.573s (vs. 0.004s).
   - *Expert Explanation:* Feasibility Pump heuristics depend on LP simplex tie-breaking orders. At certain dimensions or thread memory shifts, the initial projection fails to satisfy diagonal constraints immediately, forcing Gurobi to generate cutting planes or explore a few branch nodes before locking onto an integer solution.

3. **Why CP-SAT Boolean Outperforms CP-SAT Integer at Extreme Scale:**

   - Table 3 reveals that at $N = 200$, CP-SAT Boolean solved in 1.75s whereas CP-SAT Integer required 17.90s. At $N = 400$ and $N = 800$, CP-SAT Boolean was the only solver besides Gurobi to survive (8.39s and 22.76s), whereas CP-SAT Integer timed out.
   - *Explanation:* The diagonal `AllDifferent` constraints span an expression domain from $-(N-1)$ to $2N-2$ (nearly twice the number of queens). CP domain filtering becomes sparse and porous at high $N$, triggering excessive integer backtracking. Conversely, the Boolean 0-1 model maps directly onto OR-Tools' internal SAT engine, activating Two Watched Literals and CDCL clause learning to aggressively prune conflicting partial assignments.

# 5 Summary and Comparative Evaluation

## 5.1 Comprehensive Synthesis of Evaluated Methods

Synthesizing the structural model properties (Table 1), the median runtimes across 5 benchmark trials (Table 2), and the extreme-scale stress testing up to $N = 800$ (Table 3), Table 4 provides a comprehensive overview of the strengths and practical constraints of each paradigm.

Table 4: Comprehensive synthesis of strengths, weaknesses, and practical limits across paradigms (incorporating data from Table 2 and Table 3)

| Method | Key Advantages | Primary Limitations |
|---|---|---|
| **SAT (Sequential)** | Linear clauses $O(m)$, high unit propagation strength, requires no commercial solver. | Complex auxiliary chaining; less intuitive than integer models; prone to TIMEOUT at $N \geq 100$. |
| **SAT (Binary)** | Most stable SAT encoding; logarithmic auxiliary bits $O(\log_2 m)$; smooth scaling (1.38s at $N = 100$). | Indirect bit propagation; unable to scale past $N \geq 400$ as shown in Table 3. |
| **SAT (Binomial)** | Highly intuitive and effortless to implement; very fast for $N \leq 16$. | Severe $O(N^4)$ clause explosion ($> 1.64$ million at $N = 100$), causing severe CPU cache misses. |
| **Pure CP** | Fully self-contained without external dependencies; instant solutions for $N \leq 12$. | Exponential time complexity $O(N!)$; completely collapses with 100% timeout at $N \geq 32$. |
| **CPLEX CP** | **Absolute Invariance**: Solve times remain virtually constant (0.028s–0.036s for $N = 4 \ldots 100$) via `AllDifferent`. | Requires proprietary IBM CPLEX Optimization Studio installation. |
| **CP-SAT (Integer)** | Elegant $O(N)$ formulation, extremely low memory footprint; highly efficient up to $N \leq 100$ (0.73s). | Times out at $N \geq 400$ due to domain dilution on diagonal expressions (see Table 3). |
| **CP-SAT (Boolean)** | Fully leverages OR-Tools' underlying SAT CDCL engine; impressively survives up to $N = 800$ (22.76s) in Table 3. | $O(N^2)$ variables and constraints, requiring higher memory than the integer formulation. |
| **Gurobi MIP** | **Unrivaled at Extreme Scale**: Solves $640,000$ binary variables at $N = 800$ in just **0.224s** (Table 3) via Primal Feasibility Heuristics. | Requires commercial or academic license; $N^2$ binary matrix incurs moderate memory usage. |

### 5.2 Practical Recommendations

- **For Standard to Moderate Dimensions ($N \leq 100$): CPLEX CP** and **CP-SAT (Integer)** represent the optimal choices due to their concise $O(N)$ formulation, near-zero RAM usage, and instant solve times. When an open-source, purely SAT-based solver is required, **SAT Binary** offers the highest predictability and safety.
- **For Extreme Dimensions ($N \geq 200$ to $N = 800$):** As validated by the stress tests in Table 3, **Gurobi MIP** (0.224s at $N = 800$) and **CP-SAT Boolean** (22.76s at $N = 800$) are the only two frameworks capable of conquering combinatorial spaces exceeding half a million binary variables without getting lost in exponential branching.
- **For Real-World Constrained Optimization (Cost vs. Feasibility): Gurobi MIP** and **CP-SAT** remain industry standards due to their seamless unification of linear objective functions and complex combinatorial constraints.

## Acknowledgments

Special thanks to Dr. To Van Khanh for helping me understand the assignment requirements more clearly.
Special thanks to Google Gemini for providing academic support (facilitating theoretical understanding of exact combinatorial solving methods) and assisting with the report writing process.

## Data Availability

All source code implementing the 12 solver configurations, automated benchmark scripts, raw data from 5 independent baseline trials and 2 extreme-scale stress tests, along with the complete LaTeX manuscript source, are openly available in the GitHub repository:
https://github.com/dxbang1383/N_Queens

## References


[1] M. Bezzel, "Schachfreund," *Berliner Schachzeitung*, vol. 3, pp. 363, 1848.

[2] J. Bell and B. Stevens, "A survey for the $N$-queens problem," *Discrete Applied Mathematics*, vol. 157, no. 1, pp. 1–29, 2009.

[3] C. Sinz, "Towards an optimal CNF encoding of boolean cardinality constraints," in *CP*, Springer, 2005, pp. 827–831.

[4] J.-C. Régin, "A filtering algorithm for constraints of difference in CSPs," in *Proceedings of the 12th National Conference on Artificial Intelligence (AAAI)*, 1994, pp. 362–367.

[5] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Available: https://www.gurobi.com

[6] L. Perron and V. Didier, "OR-Tools," Google, 2024. [Online]. Available: https://developers.google.com/optimization

[7] IBM Corp., "IBM ILOG CPLEX Optimization Studio User's Manual," Version 22.1, 2022.